PROTOCOL & CODELOCKED

presents

Mega Hackathon 2026

Cali_Burrito

48

PROBLEM STATEMENT 3

March 13, 2026

INTRODUCTION

Understanding the Problem

Manual document processing leads to severe bottlenecks, extremely long queues, and an increased risk of human error.

Border agents are overwhelmed by complex, constantly updating visa policies and must verify identities across fragmented, disconnected databases.

The current system is slow, stressful, and vulnerable to security oversights.

Identifying Key Challenges

A comprehensive analysis of the core challenges we face.

Overview

The problem involves a critical evaluation of existing digital interfaces and their limitations in delivering user-centric solutions.

Challenges

Our analysis reveals significant usability issues, inadequate functionality, and poor user engagement, which hinder effective interaction and overall user satisfaction with the current digital systems.

PROPOSED SOLUTION

Our Innovative Approach

Solution Diagram

Overview of Architecture

1. <u>The Input (Streamlit Frontend)</u>
The border officer types in the traveler's ID number.
The officer uploads a photo of the traveler's physical passport or ID card.

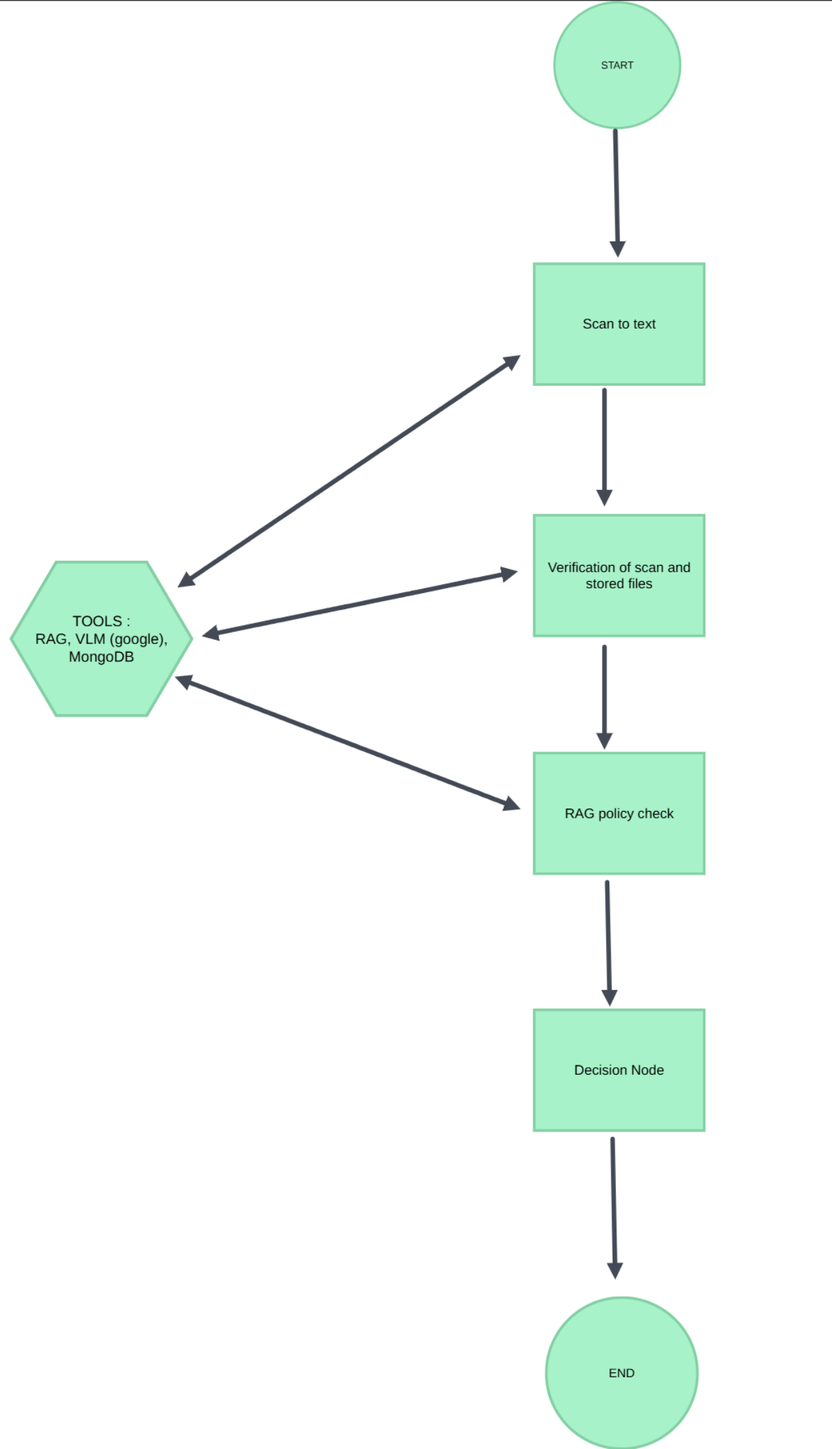
2. <u>Extraction Node (VLM)</u>
The Vision-Language Model scans the uploaded image.
It extracts key details (Name, Passport Number, Nationality, Expiry Date) into a clean JSON format.

3. <u>Profile Retrieval Node (MongoDB)</u>
The graph takes the officer's typed ID number and queries your MongoDB (or local JSON/CSV file).
It pulls the traveler's existing file (past entry logs, watchlist status).

4. <u>Verification Node (Python Logic)</u>
The system cross-references the VLM's extracted text with the database profile.
It immediately flags any discrepancies (e.g., the scanned passport number doesn't match the database).

5. <u>Policy Check Node (Vector DB & RAG)</u>
The graph queries your Vector DB containing official border laws.
It checks rules relevant to the traveler (e.g., "Does a citizen of [Nationality] need a transit visa?").

6. <u>Decision Node (LLM)</u>
An LLM evaluates all the gathered data and comes to a conclusion.



Tech Stack Overview

Frontend: Streamlit (Provides the officer's dashboard for ID input and photo uploads).

Orchestration: Python / LangGraph (Manages the state and transition between the Extraction, Retrieval, and Decision nodes).

Database (Structured): MongoDB (Stores traveler profiles and historical logs).

Database (Unstructured): FAISS (Vector Database for storing and searching border law documents).

Vision Model: Google Gemini (VLM) (Handles the "Scan to Text" extraction of physical IDs).

Reasoning Model: GPT-4o or Gemini 1.5 Pro (Powers the RAG policy check and final decision logic).

Impact and Benefits

Benefit	Impact
Operational Efficiency	Reduces manual entry time by up to **80%** using VLM automated extraction.
Enhanced Security	Real-time cross-referencing between physical IDs and MongoDB logs eliminates clerical mismatch errors.
Regulatory Compliance	FAISS + RAG ensures that officers always act on the *most current* version of border laws without needing to memorize them.
Cost Effective	Leveraging FAISS (Open Source) and local processing reduces the need for expensive, high-bandwidth cloud infrastructure.

Thank You!